IoT-APIScanner: Detecting API Unauthorized Access Vulnerabilities of IoT Platform

Yilian Li[1,2] , Yiyu Yang[2], XiaoYu[3,2], Ting Yang[4,2], Lihua Dong[1], Wenjie Wang[2,*]

[1] School of Communication Engineering, Xidian University, Xi'an, China
[2] National Computer Network Intrusion Protection Center, University of Chinese of Academy of Sciences, Beijing, China
[3] School of Cyber Engineering, Xidian University, Xi'an, China
[4] College of Information Science and Engineering, Hebei University of Science and Technology, Shijiazhuang, China
*Corresponding author: Wenjie Wang
wangwj@ucas.ac.cn

***Abstract*—The Internet of Things enables interaction between IoT devices and users through the cloud. The cloud provides services such as account monitoring, device management, and device control. As the center of the IoT platform, the cloud provides services to IoT devices and IoT applications through APIs. Therefore, the permission verification of the API is essential. However, we found that some APIs are unverified, which allows unauthorized users to access cloud resources or control devices; it could threaten the security of devices and cloud. To check for unauthorized access to the API, we developed IoT-APIScanner, a framework to check the permission verification of the cloud API. Through observation, we found there is a large amount of interactive information between IoT application and cloud, which include the APIs and related parameters, so we can extract them by analyzing the code of the IoT application, and use this for mutating API test cases. Through these test cases, we can effectively check the permissions of the API. In our research, we extracted a total of 5 platform APIs. Among them, the proportion of APIs without permission verification reached 13.3%. Our research shows that attackers could use the API without permission verification to obtain user privacy or control of devices.**

Keywords—Web API, IoT Security, Unauthorized Access

I. INTRODUCTION

The IoT cloud platform consists of three parts: the IoT application(IoT APP), devices, and the cloud. IoT APP can access the device through the cloud or the local area network. With the development of the Internet of Things, more and more platform vendors canceled LAN communications. IoT APP gradually began to access devices only through the cloud, and the cloud became the link between the IoT APP and devices. Therefore, cloud security became more important. At the same time, because the cloud API is the entrance for the IoT APP and devices to access the IoT cloud platform resources and also the exit for the cloud platform to return the response. So, it is meaningful to study the security of API.

Now, some researches have focused on the security of IoT cloud platforms. For example, Jiansong Zhang et al. [1] found that the authentication mechanism of the existing platform has security vulnerabilities, and attackers can launch device forgery attacks, leading to leakage of WiFi passwords. Besides, some literature has analyzed the privacy and security of cloud platforms [2-4]. However, the goals of these studies are focus on the security of the internal design of the cloud platform, with little attention to API services.

Unlike ordinary mobile APPs, IoT APP can not only access resources in the cloud, but also access IoT devices through APIs. Normally, After receiving the APP request, the cloud API service must perform identity authentication and authorization verification. Only after these two checks pass, the cloud would return the corresponding resources to the APP or convert the request into a device instruction that would be sent to the device. Therefore, API plays an essential role in the security of cloud platforms and devices. However, in our research, we found that some attackers can bypass the verification in the cloud by a specific way to achieve the API unauthorized access. This unauthorized access would cause serious device security issues. Nevertheless, there is little research on analyzing the access control security of cloud platforms and devices with cloud platform APIs as the analysis object, and there is also a lack of tools to detect unauthorized access vulnerabilities caused by unauthorized authorization of these APIs.

To systematically detect unauthorized access vulnerabilities in the API of the IoT, we have studied several popular IoT platforms. First, we collected the APIs and the parameters provided in the current popular IoT platform APP. Then, we implemented an IoT-APIScanner，a framework for detecting API unauthorized access vulnerabilities, which can visit and detect these APIs in the APP for unauthorized access vulnerabilities. Based on the results of the detection, we have successfully discovered and categorized API unauthorized access vulnerabilities in 5 IoT APPs. Finally, we found the cause of the vulnerability. In summary, our contributions are as follows:

- We provide new insight into the IoT platform, investigate the security issues of the IoT platform for analysis objects through the analysis API security.

Authorized licensed use limited to: Middlesex University. Downloaded on November 03,2020 at 03:09:11 UTC from IEEE Xplore. Restrictions apply.

- We developed IoT-APIScanner, a framework for detecting unauthorized access to the Internet of Things API.
- We successfully found 21 APIs in 5 IoT APPs that have unauthorized access vulnerabilities. We use real smart devices to assess the harm of unauthorized access vulnerabilities.

II. Background

In this section, we introduced the IoT Interaction Model based on API-Service. Then we showed the API unauthorized access vulnerability exists in this model.

A. Smart Home Interaction Model Based On API-Service

The IoT interaction model usually includes three roles: device, IoT application (IoT APP), and cloud. Devices including hardware entities and software programs running on hardware. IoT APP is an application running on a smartphone or other mobile device. The cloud represents the applications deployed on the cloud platform, which mainly provide functions such as account login, device monitoring, device control, et al. Users use IoT APP to access device information and control instructions through API provided by the smart home platform. Fig.1 shows the interaction model of smart home. This model has been adopted by most smart home cloud platforms (such as SmartThings [5], Tuya [6], Alibaba [7], et al.).

IoT APP communicates with the cloud server through APIs provided by the cloud. The communication protocol usually uses HTTPS and OAuth[8] to verify identity between server and application and protect the privacy and integrity of the exchanged data. Therefore, the cloud needs to verify the identity of the API access object strictly and prevent illegal users from accessing legitimate device resources. IoT-APIScanner runs in SmartPhone, and intercepts and replaces API requests issued by IoT APP to achieve unauthorized verification of API.

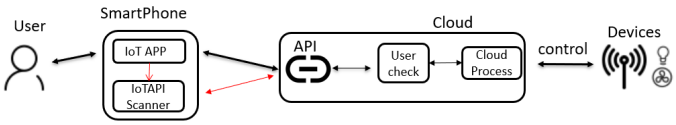

Fig.1. The interaction model of smart home.

B. API Unauthorized Vulnerability

From the previous section, we know the API is the entrance for APPs to access cloud resources. The complete API consists of API address + API input parameters + API response results. Take the Xiaomi platform API as an example. The API to get its own devices is shown below: its API address is https://api.io.mi.com/app/v2/home/device_list_page, and the input parameter is {"dids": ["111111"]}, where did is the device identification number. The response result is the basic information of the specific devices. Developers usually modify the API interface according to the addition and deletion of APP functions. Ideally, the cloud needs to verify all the APIs it provides strictly. Due to the rapid development of smart home and Android APP, we found that some APIs are incomplete or no verification. These will cause the attacker can obtain device information and even control the device. For example, we discovered that Joylink API for setting scheduled tasks allows any user to access, and the cloud does not verify the identity of any user on this API. Therefore, an attacker can use this API to implement remote control of devices. We refer to the problem of no or incomplete verification in the API as an API unauthorized access vulnerability.

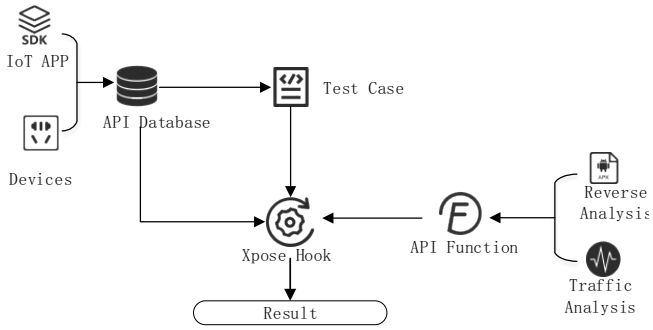

Fig.2. IoT-APIScanner framework

III. Detection and Implementation of IoT-APIScanner

In this section, we introduce the design ideas and implementation details of IoT-APIsScanner. As shown in Fig. 2, IoT-APIScanner is a framework to analyze the problem of API unauthorized access vulnerability in the android platform. IoT-APIScanner needs to obtain all the APIs and corresponding parameters allowed in the IoT APP. Then it visits and inspects all APIs in turn. Finally, we analyzed the problem of unauthorized access to the API based on the detection results.

A. API and API parameter extraction

To detect all APIs, we need to extract the API and its corresponding parameters. Some researchers have previously implemented API extraction and analysis for ordinary Mobile APPs [6-8]. However, according to actual analysis, we found that most of the Internet of Things APPs are closed-source, and they use packer technology and obfuscation technology to prevent attackers from decompiling and acquiring APP logic. These make most API extraction tools unusable.

Therefore, for these IoT Apps, we combine UI automated testing and network traffic analysis tools to extract API information. First, we need to analyze the program code in the IoT APP and find all the UI components that cause API calls. Next, we use UIAutomator to simulate component interactions, such as turning on lights, unbinding devices, et al. Thus, the automatic testing can trigger the API network request event. Then, we use the packet capture tool Fiddler [9] to capture all the requests, and then extract all the requests into API and corresponding parameters through the python script. Figure 3 shows some APIs and corresponding parameters extracted by

Authorized licensed use limited to: Middlesex University. Downloaded on November 03,2020 at 03:09:11 UTC from IEEE Xplore. Restrictions apply.

the Xiaomi platform. Finally, we mark all the device-related APIs with the feature value of the device and exclude user-related APIs such as login and registration.

```
{from":"4","id":4666,"method":"get_prop","params":["ar
ming","night_light_rgb","rgb"]}
{"from":"4","id":631476,"method":"remove_device","para
ms":["lumi.158d0003f83a4e"]}
{"from":"4","id":4602,"method":"set_default_music","para
ms":[0,"1"]}
{"from":"4","id":2201854,"method":"miIO.restore","para
ms":[]}
```

Fig.3. API format of MiJia APP

B. Static Analysis

Our detection method is to replace the normal request with the test request for the device ID and operation instruction sent by the API when the APP sends the operation. Therefore, we first need to find a suitable function replacement point.

We use the static analysis of Jeb [10] and Jdax [11] to find all valid paths from the device ID to the network send function. Then we find reasonable Hook points through these paths. We found that for the simplicity and convenience of the code in the APP, the APP usually encapsulates the device in a fixed class. For example, Xiaomi uses the Device class to encapsulate various attributes of the device. The Device class contains the basic status information of the device (such as device ID, device model, device type, et al.) and its functions to access this information. We use the function to obtain the device ID as the entry point, and use the network send data functions such as *URL.openConnection()* and *Socket.getOutputStream()* as the endpoint to generate a complete control flowchart. And select the most suitable Hook point in the path.

C. API Test

We use the Xposed[12] Hook framework to implement dynamic testing of the API. In the *beforeHook* function, the original field of the function is modified to the test API and corresponding parameters. Because the communication between the APP and the device is based on HTTP / HTTPS, we use Fiddler to build a Man in the middle attack to obtain the response obtained by the test case.

Fiddler is an HTTP protocol debugging proxy tool. Through Fiddler, we can record all HTTP communication between the APP and the cloud. We use Fiddler to grab the network request packet we sent and the corresponding results. And use the Python script to extract the request packet and corresponding response we sent. Finally, an unauthorized vulnerability analysis is performed based on the response.

IV. Evaluation

In this section, we analyze APPs for 5 IoT platforms, including MiJia[13], Joylink[14], Alink, SmartThings, TuyaSmart.We extract 157 APIs through our tool, and then we use the IoT-APIScanner to check whether the API has unauthorized vulnerabilities. Finally, we described two cases that show how we verified the API unauthorized access vulnerability based on the output.

A. Identifying Unauthorized Vulnerabilities

TABLE I. API Unauthorized Vulnerability Analysis Results

IoT platform	*IoT APP*	*Number of extraction APIs*	*Number of Uncheck APIS*	*Number of APIs with security risks*	*Proportion of uncheck APIs*
MiJia	MiHome	58	3	3	5.2%
Joylink	JD Whale	58	11	8	19.0%
Alink	AliSmart	17	3	2	17.6%
Smartthings	Smartthings	10	2	1	20%
TuyaSmart	TuyaSmart	14	2	0	14.3%

We classify the output of IoT-APIScanner according to the smart home platform, as shown in Table 1. The results show that IoT-APIScanner identified a total of 21 APIs with verification problems (i.e., unauthorized access vulnerabilities). After manual verification, we found some API unauthorized access vulnerabilities can be exploited.

B. Impact analysis of API unauthorized exploits

In this section, we used BroadLink smart plug for the Joylink platform and smart plug for Alink to demonstrate the API unauthorized access vulnerabilities and their effects.

Joylink: According to the experimental results, a total of 11 API unauthorized access vulnerabilities are found in the APP-cloud interaction of Joylink, among which the more important ones are related to timing tasks. Setting timing tasks is a particular function of operating devices provided by JD Whale. Take adding timing tasks as an example, users set the operating details and execution time in advance in the APP interface, and the cloud will automatically send the operation instructions to the device at the scheduled time. There are three types of timing task operation: add, query, delete. Each type needs to provide parameters such as user and device identity information. After the cloud receives the request parameters, the cloud API should check whether the user has operation permissions and must refuse to perform requests from users who are out of authorized scope. However, these APIs are not to do so. An example of an unauthorized accessing device through the timing task APIs is shown below. After the device owner binds a BroadLink smart plug, he shares the operation privileges with the guests, who can obtain the device identity information(such as DeviceId) technically. The owner revoked the shared operation permissions, guests at this time no longer have any operating privileges to the device. However, guests could still directly issue a query or delete request to the timing task API with the parameters of device identity information. Finally, guests could get BroadLink smart plug's timing task list, and remove all tasks at will.

Alink: We found that the API of *mtop.alink.app.core.device.get.info* and the API of *mtop.alink.app.core.devices.index.query* in Alink are used by users to obtain device information, and their parameters are corresponding device

Authorized licensed use limited to: Middlesex University. Downloaded on November 03,2020 at 03:09:11 UTC from IEEE Xplore. Restrictions apply.

identification numbers. However, we found that Alibaba Cloud did not perform any identity verification on the interface. Therefore, we can use this API to get device privacy. For example, after the device owner binds Alibaba's device and shares the operation permissions to the guest user, the guest user can obtain the device identification number. Then the host revokes the guest's operation permission. The visitor no longer has any permission to operate the device. Visitors can still access *mtop.alink.app.core.device.get.info* API to monitor the status of the smart device at this time. At the same time, because the device identification number is a fixed value, an attacker can obtain all the devices and their previous operating states through brute force cracking.

V. Related work

In this section, we describe the related research on IoT security from two parts: IoT security and APP API security analysis. Next, we compared the existing Web API testing tools with API-Scanner and described the difference between current research and IoTAPI-Scanner.

A. IoT platform security

Fernandes et al. [15]found that the current logic design of the Trigger-Action platform has the problem of over-authorization in OAuth tokens, which brings long-term and large-scale security risks to user privacy and data security. Somerset Recon inspected and found multiple vulnerabilities in Hello Barbie[16]. They found that the cloud did not authenticate the firmware download, and it has multiple cross-site scripting vulnerabilities that allowed attackers to enumerate usernames violently. What is more, they found cookies provided by the cloud never expire. Obermaier et al. [17] audited the cloud endpoints of surveillance cameras, and they found attackers could inject lenses, trigger false alarms, and perform a denial of service attacks on the camera system.

B. APP API analysis

There are many studies on the API analysis of the Android APP—for example, Pascal et al.[18] developed a static analysis tool to identify the API of mobile applications and extract communication information such as API and related JSON headers. It also found security problems such as injection attacks and unnecessary server configuration leaks in some APIs. Mendoza et al. [19] developed a tool for extracting API requests from the APP, and they analyzed the APP through the black box test to verify whether the input verification of the API service is correctly implemented. The results show that about 40% of the APIs existing Hijacking vulnerability. It would threaten the privacy of millions of users. What's more, we found that there are many tools for security analysis of the APIs of the Android APP[20-22]. However, these tools are difficult to run when we try to apply them to the IoT APP. It probably due to the IoT APPs are closed source, obfuscated, and have a large amount of code.

C. Web API testing tools

Nowadays, there are many Web API testing tools such as SoupUI[23], PosetMan[24], Jmeter[25]. They can also be used to test APP API security issues. These tools are often used for API performance testing, functional testing, input verification, et al. They detect the security problems of the API by sending requests outside the APP, so they need to obtain the API address and request parameters of the test object first. However, for the current IoT APPs, most of them use encryption and signature to verify API network requests, and most of the APP's signature and encryption codes are confused. Some IoT APPs put the signature and encryption codes in the so library, which makes Web API tools unable to analyze the API of the IoT APP directly. Unlike these tools, IOT-APIScanner is based on Xposed dynamic replacement technology, which converts the normal request into a test request before the network request data being generated, so that we can solve the problem of signature and encryption.

VI. Discussion

A. Mitigation

The cloud should strengthen the verification of the API. When adding an API, it is necessary to verify the security of the API before commercializing it. At the same time, when the APP new functions and features, it is essential to reasonably check and analyze, prevent security issues such as the API not matching the new service.

B. Limitation

First of all, the detection method of IoT-APIScanner is based on Xposed Hook technology, but the specific position of the Hook needs to be found by manual reverse analysis. Therefore, if researchers use IoT-APIScanner, they need to reverse the APP and analyze the appropriate Hook point manually. Later, we could use the static analysis method to achieve the function of automatic positioning Hook. It would reduce the time spent on manual analysis.

Secondly, we only analyzed the API access between APP and cloud, and the problem of API between device and cloud is not explained. We would also pay attention to the API call between the cloud and the device and explore its potential security risks in future work.

Finally, we found that not all unauthenticated APIs have unauthorized access. Some APIs are designed to allow all users to access them, such as obtaining the latest version of the APP and checking for device updates. Therefore, after finding out all the unverified APIs, it needs to be manually identified again to determine whether these APIs have the problem of unauthorized access. Later we need to test more IoT platforms and explore what security issues the API.

VII. Conclusion

This article explains the security problems of the IoT platform from a new insight. Furthermore, we propose the IoT-APIScanner, which uses API as the detect object, identified unauthorized access vulnerabilities in the API. The idea of IoT-APIScanner is based on static analysis to identify the API and dynamic detection by Xposed. With the help of IoT-APIScanner, our evaluation shows that IoT-APIScanner can effectively help the cloud platform to check the API's security problems. We

Authorized licensed use limited to: Middlesex University. Downloaded on November 03,2020 at 03:09:11 UTC from IEEE Xplore. Restrictions apply.

have shown the impact of API unauthorized access vulnerabilities on smart devices. Besides. In the end, we put forward some defensive suggestions on how to prevent it effectively.

ACKNOWLEDGMENT

This work is supported by The Nation Key R&D Program China(2018YFB0804701), The National Natural Science Foundation of China(No.U1836210, No.61572460).

REFERENCES

[1] J. Zhang, Z. Wang, Z. Yang and Q. Zhang, "Proximity based IoT device authentication," IEEE INFOCOM 2017 - IEEE Conference on Computer Communications, Atlanta, GA, 2017, pp. 1-9.

[2] Wang, Q. , Hassan, W. U. , Bates, A. , & Gunter, C. . (2018). Fear and Logging in the Internet of Things. Network and Distributed System Security Symposium.

[3] Milijana Surbatovich, Jassim Aljuraidan, Lujo Bauer, Anupam Das, and Limin Jia. 2017. Some Recipes Can Do More Than Spoil Your Appetite: Analyzing the Security and Privacy Risks of IFTTT Recipes. In Proceedings of the 26th International Conference on World Wide Web (WWW '17). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 1501–1510.

[4] H. Luo, R. Wang and X. Li, "A Rule Verification and Resolution Framework in Smart Building System," 2013 International Conference on Parallel and Distributed Systems, Seoul, 2013, pp. 438-439.

[5] "Samsung SmartThings," https://www.smartthings.com/,2020.2.

[6] "TuyaSmart", 2, 2020.

[7] "Alink", https://smart.yun.taobao.com/,2020.2.

[8] E. Hammerlahav E . The OAuth 2.0 Protocol. 2010.

[9] "Fiddler", https://www.telerik.com/fiddler,:2020-2.

[10] "Jeb", https://www.pnfsoftware.com/ ,2020-2

[11] "Jdax" ,https://github.com/skylot/jadx, 2020-2.

[12] "Xpose", https://github.com/rovo89/XposedTools, 2020-2.

[13] "MiJia", https://home.mi.com/index.html , 2020-2.

[14] "Joylink", http://smartdev.jd.com/,2020-2.

[15] Fernandes, Earlence & Rahmati, Amir & Jung, Jaeyeon & Prakash, Atul. (2018). Decentralized Action Integrity for Trigger-Action IoT Platforms. 10.14722/ndss.2018.23121.

[16] A. Blaich and A. Hay, Hello Barbie Initial Security Analysis. https://static1.squarespace.com/static/543effd8e4b095fba39dfe59/t/56a66d424bf1187ad34383b2/1453747529070/HelloBarbieSecurityAnalysis.pdf, 2018.

[17] Johannes Obermaier and Martin Hutle. 2016. Analyzing the Security and Privacy of Cloud-based Video Surveillance Systems. In Proceedings of the 2nd ACM International Workshop on IoT Privacy, Trust, and Security (IoTPTS '16). Association for Computing Machinery, New York, NY, USA, 22–28.

[18] Gadient P , Ghafari M , Nierstrasz O . Web APIs in Android through the Lens of Security. https://arxiv.org/pdf/2001.00195.pdf , 2020.

[19] A. Mendoza and G. Gu, "Mobile application web API reconnaissance: Web-to-mobile inconsistencies & vulnerabilities," in 2018 IEEE Symposium on Security and Privacy (SP). IEEE, 2018, pp. 756–769.

[20] M. Rapoport, P. Suter, E. Wittern, O. Lhotak, and J. Dolby, "Who ´you gonna call?: Analyzing web requests in Android applications," in Proceedings of the 14th International Conference on Mining Software Repositories, ser. MSR '17. Piscataway, NJ, USA: IEEE Press, 2017,pp. 80–90.

[21] Yajin Zhou, Lei Wu, Zhi Wang, and Xuxian Jiang. 2015. Harvesting developer credentials in Android apps. In Proceedings of the 8th ACM Conference on Security & Privacy in Wireless and Mobile Networks (WiSec '15). Association for Computing Machinery, New York, NY, USA, Article 23, 1–12.

[22] C. Zuo and Z. Lin, "Smartgen: Exposing server URLs of mobile apps with selective symbolic execution," in Proceedings of the 26th International Conference on World Wide Web, ser. WWW '17.Republic and Canton of Geneva, Switzerland: International World Wide Web Conferences Steering Committee, 2017, pp. 867–876.

[23] "SoupUI", https://www.soapui.org/,2020-5.

[24] "PostMan", https://www.postman.com/,2020-5.

[25] S. Shenoy, N. A. A. Bakar and R. Swamy, "An Adaptive Framework for Web Services Testing Automation Using JMeter," 2014 IEEE 7th International Conference on Service-Oriented Computing and Applications, Matsue, 2014, pp. 314-318, doi: 10.1109/SOCA.2014.36.

Authorized licensed use limited to: Middlesex University. Downloaded on November 03,2020 at 03:09:11 UTC from IEEE Xplore. Restrictions apply.